

MTH2526-B25

QR Code Generator Service

Bulut Mimarilerinde Test Mühendisliği – Dönem Projesi

Büşra Ay • 170620002

Bilgisayar Mühendisliği Bölümü

2025–2026 Bahar Yarıyılı

Eğitmen: Büşra Ayaksız

Nelerden Bahsedeceğiz?

1. Projenin Amacı ve Kapsamı
2. Sistem Mimarisi
3. Teknoloji Yığını
4. Test Stratejisi (Unit · Integration · E2E)
5. CI/CD Pipeline (GitHub Actions)
6. Container ve Kubernetes Dağıtımı
7. AWS / LocalStack Entegrasyonu
8. Performans Testi (k6)
9. Monitoring: Prometheus & Grafana
10. Karşılaşılan Zorluklar ve Çözümler
11. Sayısal Sonuçlar ve Değerlendirme
12. Sonuçlar

Projenin Amacı ve Kapsamı

Amaç

- Kullanıcıdan metin/URL olarak PNG ve SVG formatında QR kod üretmek
- Ders kapsamındaki test mühendisliği araçlarını endüstri standartlarında uygulamak
- Bulut tabanlı mikroservis geliştirmek
- Coverage $\geq$ %70, multi-stage Docker, Kubernetes deploy ve monitoring

Kapsam

- Konu 35: QR Code Generator Service
- Bireysel çalışma
- FastAPI REST API + Web arayüzü
- PostgreSQL metadata depolama
- LocalStack S3 dosya depolama
- GitHub Actions CI/CD pipeline
- Minikube Kubernetes dağıtımı
- Prometheus + Grafana monitoring

Sistem Mimarisi



Veri Akışı:

1. Kullanıcı arayüzden veya API üzerinden metin/URL gönderir
2. FastAPI isteği Pydantic ile doğrular, QR kodunu PNG ve SVG olarak üretir
3. Dosyalar LocalStack S3'e (veya fallback olarak local_storage'a) yüklenir
4. Metadata (UUID, etiket, checksum, hit_count) PostgreSQL'e kaydedilir
5. İndirme isteğinde dosya depolamadan okunup Content-Disposition: attachment ile sunulur
6. Prometheus /metrics endpoint'inden metrikleri scrape eder, Grafana ile görselleştirilir

Çözüm Mimarisi: Araçlar ve Stratejiler

Katman	Teknoloji	Gerekçe
API Framework	FastAPI	Asenkron, tip güvenli, otomatik Swagger
Veritabanı	SQLAlchemy + PostgreSQL	İlişkisel metadata, container uyumu
Depolama	LocalStack S3 (boto3)	AWS S3 davranışını yerelde simüle
Unit Test	pytest + Factory Boy + Faker	Fixture, parametrizasyon, hızlı feedback
Integration	testcontainers + LocalStack	Gerçek container'larla doğrulama
E2E Test	Playwright (Chromium)	Tarayıcı seviyesinde kullanıcı akışı
Perf Test	k6 (Grafana)	Yük testi, p95 latency ölçümü
Monitoring	Prometheus + Grafana	Metrik toplama ve dashboard
CI/CD	GitHub Actions	Otomatik lint → test → deploy pipeline
Container	Docker (Multi-Stage)	Hafif, güvenli imaj (~231 MB)
Orkestrasyon	Kubernetes (Minikube)	Self-healing, liveness/readiness probe

Test Stratejisi: Unit & Integration



Unit Testler (17 test)

- pytest framework ile 17 birim testi
- Factory Boy + Faker: rastgele, gerçekçi test verisi üretimi
- Dış bağımlılıklar mock'lanmış:
 - Veritabanı: InMemory SQLite
 - Depolama: fake_storage fixture
- Çalışma süresi: < 1 saniye
- QR üretici, servis mantığı, route handler ve config doğrulama testleri
- conftest.py ile merkezi fixture yönetimi



Integration Testler (3 test)

- Mock yerine gerçek container'lar
- Testcontainers PostgreSQL:
 - Docker'da geçici postgres:16 imajı
 - Tablo oluşturma ve CRUD doğrulaması
 - Silme tetikleyicileri testi
- LocalStack S3 Entegrasyonu:
 - Bucket oluşturma → dosya yükleme
 - Geri okuma ve SHA-256 bütünlük kontrolü doğrulaması
- Gerçek ortam davranışını izole test eder

Test Stratejisi: E2E & Coverage



E2E Testler – Playwright (5 test)

- Playwright Headless Chromium ile tarayıcı seviyesinde otomasyon
- Test senaryoları:
 - Ana sayfa yüklenme kontrolü
 - QR kod oluşturma form akışı
 - PNG önizleme doğrulaması
 - PNG/SVG indirme (attachment header)
 - İndirme sayacı (hit_count) artışı
- Bug Fix: Content-Disposition inline → attachment + download attribute eklendi
- Test'te page.request.get kullanıldı



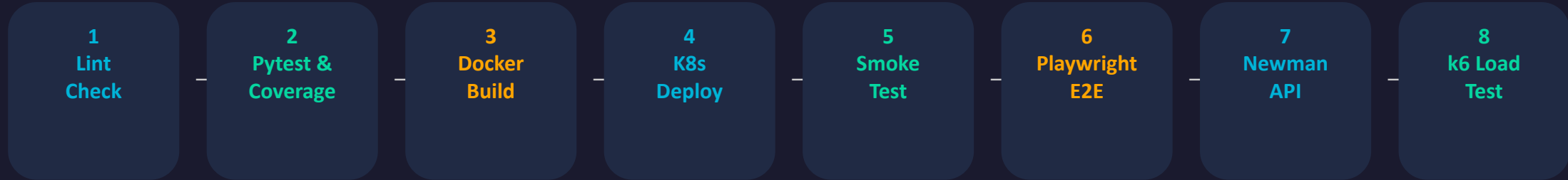
Kod Kapsama (Coverage)

%88.19

Hedef: $\geq$ %70  Başarılı

- pytest-cov ile ölçüm
- src/ altındaki tüm modüller kapsanır
- Servis, route ve config katmanları
- CI pipeline'da --cov-fail-under=70

CI/CD Pipeline – GitHub Actions



Pipeline Detayları (.github/workflows/ci.yml):

- Her push ve pull request'te otomatik tetiklenir
- Ruff ile statik kod analizi (lint) yapılır
- Unit ve integration testler coverage ile çalıştırılır (fail-under: %70)
- Multi-stage Docker imajı build edilir
- Minikube cluster ayağa kaldırılır, imaj yüklenir, K8s manifestleri uygulanır
- Smoke test: /health endpoint'ine curl ile sağlık kontrolü
- Playwright E2E testleri Minikube URL'si üzerinde çalışır
- Newman ile Postman collection çalıştırılır
- k6 yük testi ile performans eşikleri doğrulanır

Container & Kubernetes Dağıtımı



Multi-Stage Dockerfile

Stage 1 – Builder:

- python:3.11-slim base imaj
- Virtual environment oluşturma
- Bağımlılıkların derlenmesi

Stage 2 – Runtime:

- Sadece venv ve src kopyalanır
- gcc, python3-dev gibi derleme araçları nihai imajda YOK
- İmaj boyutu: ~231 MB
- Uvicorn ile port 8000'de çalışır



Kubernetes Manifestleri

k8s/ dizininde 3 manifest dosyası:



configmap.yaml

- DB URL, S3 bucket, region ayarları



deployment.yaml

- App + PostgreSQL + LocalStack pod'ları
- Liveness & Readiness probe'ları
- Self-healing (otomatik yeniden başlatma)



service.yaml

- ClusterIP ve NodePort tanımları

AWS / LocalStack Entegrasyonu

LocalStack ile AWS S3 Simülasyonu

- LocalStack 3.8 sürümü ile S3 API'si yerelde çalıştırılıyor
- qr-code-assets bucket'ı oluşturularak PNG ve SVG dosyaları depolanıyor
- boto3 Python SDK ile S3 operasyonları: put_object, get_object, delete_object
- docker-compose.yml'de localstack servisi port 4566'da tanımlı

Fallback Mekanizması (Hata Toleransı):

- StorageService sınıfı başlangıçta S3'e bağlanmayı dener
- BotoCoreError veya ClientError alınırsa → otomatik local_storage/'a geçiş
- _switch_to_local() metodu ile yerel dosya sistemi aktifleşir
- Geliştirme ve demo ortamlarında dış servis bağımlılığı olmadan çalışır

Entegrasyon Testleri ile Doğrulama:

Performans Testi – k6



Test Konfigürasyonu

Senaryo: perf/load-test.js

- 10 eşzamanlı sanal kullanıcı (VUs)
- 30 saniye sürekli yük
- POST /api/v1/qrcodes endpoint'ine
her iterasyonda yeni QR kod oluşturma

Eşik Hedefleri (Thresholds):

- http_req_failed < %1 (hata oranı)
- http_req_duration p(95) < 800ms



Sonuçlar

262.72 ms

p95 Latency (Hedef: < 800ms) ✓

%0.00

Hata Oranı (Hedef: < %1) ✓

Toplam İstek: 251

Monitoring – Prometheus & Grafana



Prometheus

- FastAPI Instrumentator ile /metrics endpoint
- HTTP istek sayısı, süre ve boyut metrikleri
- prometheus.yml: 15 saniyede bir scrape
- docker-compose.yml'de prom/prometheus:v2.54.1



Grafana Dashboard

- grafana/grafana:11.2.0 imajı
- Otomatik provisioning: datasource + dashboard
- Admin/admin ile erişim (port 3000)
- JSON tabanlı dashboard tanımı



3 Anlamlı Panel

Panel 1 – Throughput (İstek Yoğunluğu):

- `rate(http_requests_total[1m])`
- Saniyede işlenen istek sayısı (RPS)

Panel 2 – Error Rate (Hata Oranı):

- HTTP 4xx ve 5xx kodlarının oranı
- Toplam isteklere göre yüzdelik hesaplama

Panel 3 – Response Latency (Gecikme):

- `histogram_quantile(0.95, ...)`
- p95 tepki süresi (milisaniye)
- Ortalama ve percentile karşılaştırması

Karşılaşılan Zorluklar & Sayısal Sonuçlar

⚠ Karşılaşılan Zorluklar

Zorluk 1: LocalStack S3 Bağlantı Kesintileri

- S3 kapalıyken API kilitleniyordu
- Çözüm: StorageService'e dinamik fallback mekanizması eklendi
- BotoCoreError → local_storage'a geçiş

Zorluk 2: Playwright İndirme Hatası

- Content-Disposition: inline tarayıcıyı yönlendirip testi çökertiyordu
- Çözüm: Header → attachment, download attribute + page.request.get

📊 Sayısal Sonuçlar

Unit + Integration Test	20	✓
E2E Test	5	✓
Kod Coverage	%88.19	✓ (≥%70)
k6 p95 Latency	262.72 ms	✓ (<800ms)
Docker İmaj	~231 MB	✓
CI/CD Pipeline	8 aşama	✓ Yeşil
Hata Oranı	%0.00	✓ (<1%)
Grafana Panel	3 panel	✓

Sonuçlar

- Test piramidine uygun 3 katmanlı strateji başarıyla uygulandı
- CI/CD pipeline tüm aşamalarıyla yeşil
- Multi-stage Docker ile güvenli ve hafif imaj
- Kubernetes'te self-healing deploy
- LocalStack ile AWS S3 entegrasyonu
- Prometheus + Grafana ile gözlemlenebilirlik
- Tüm rubric kriterleri karşılandı

Teşekkürler